

ORACLE SQL

Theory + Query Book

A very easy, beginner friendly guide with examples and analogies

What is inside

1. SQL Basics: SELECT, WHERE, AND/OR/NOT, BETWEEN, IN, LIKE, IS NULL, ORDER BY, DISTINCT
2. Aggregate Queries: COUNT, SUM, AVG, MAX, MIN, GROUP BY, HAVING
3. Joins: Inner, Equi, Natural, Self, Outer, 3+ tables
4. Subqueries: single-row, multi-row, IN, ANY, ALL, correlated, in FROM, in SELECT
5. Set Operations: UNION, UNION ALL, INTERSECT, MINUS
6. Oracle Special: NVL, DECODE, CASE, SYSDATE, TO_DATE, TO_CHAR, string and date functions
7. Cheat sheet and practice questions

Contents

Contents	2
Before We Start: Our Practice Tables	4
Key words	4
Table 1: DEPARTMENTS	4
Table 2: EMPLOYEES	4
Table 3: PROJECTS	4
Script to create the tables (run this in Oracle)	5
Oracle rules to remember	5
SQL Basics	6
1.1 SELECT ... FROM	6
Column alias and simple maths	6
1.2 WHERE (filter rows)	7
1.3 AND, OR, NOT	7
Order matters: AND runs before OR	8
1.4 BETWEEN	9
1.5 IN	9
1.6 LIKE (pattern search)	9
1.7 IS NULL / IS NOT NULL	10
1.8 ORDER BY (sorting)	11
1.9 DISTINCT (remove duplicates)	12
Aggregate Queries	13
2.1 COUNT, SUM, AVG, MAX, MIN	13
COUNT in three flavours	13
2.2 GROUP BY	13
2.3 HAVING (filter the groups)	14
The order Oracle runs a query (very important!)	15
Joins	16
Table alias (a short nickname)	16
3.1 Inner join	16
3.2 Equi join (the old Oracle style)	17
3.3 Natural join	17
3.4 Self join	18
3.5 Outer joins	18
Left outer join: keep everything from the left table	18

Right outer join: keep everything from the right table	19
Full outer join: keep everything from both tables	19
Oracle old style: the (+) sign	19
3.6 Joining 3 or more tables	19

Subqueries 21

4.1 Single-row subquery	21
4.2 Multi-row subquery with IN	21
4.3 ANY and ALL	22
4.4 Correlated subquery	23
EXISTS: a popular correlated subquery	23
4.5 Subquery in FROM (inline view)	24
4.6 Subquery in SELECT (scalar subquery)	24

Set Operations 25

Rules	25
5.1 UNION	25
5.2 UNION ALL	25
5.3 INTERSECT	26
5.4 MINUS (Oracle only word)	26

Important Oracle-Specific Things 27

6.1 NVL (replace NULL)	27
6.2 DECODE (Oracle's if-else)	27
6.3 CASE (the modern, powerful if-else)	28
6.4 SYSDATE and DUAL	28
6.5 TO_DATE (text to date)	29
6.6 TO_CHAR (date or number to text)	29
6.7 String functions	30
6.8 Date functions	31

Cheat Sheet and Practice 33

Full query pattern	33
Quick memory tips	33
Practice questions (with answers)	33

Before We Start: Our Practice Tables

Every example in this book uses the same three small tables. Learn them once, and every query will feel familiar. A **database** is like a folder. A **table** is like one Excel sheet inside that folder. A **row** is one record (one person). A **column** is one type of information (like name or salary).

ANALOGY: Think of a company office. One sheet lists departments, one sheet lists employees, one sheet lists projects. They are kept in separate sheets so we do not type the same department name again and again.

Key words

- **Primary key:** a column whose value is different for every row (like emp_id). It identifies one row.
- **Foreign key:** a column that points to another table. Example: dept_id in EMPLOYEES points to DEPARTMENTS.
- **NULL:** means "no value" or "unknown". It is not zero and not a blank space.

Table 1: DEPARTMENTS

DEPT_ID	DEPT_NAME	LOCATION
10	Sales	Mumbai
20	IT	Pune
30	HR	Delhi
40	Finance	Chennai

Table 2: EMPLOYEES

EMP_ID	EMP_NAME	JOB	SALARY	COMMISSION	HIRE_DATE	MANAGER_ID	DEPT_ID
101	Asha	MANAGER	9000	(null)	10-JAN-2015	(null)	20
102	Ravi	DEVELOPER	6000	(null)	15-MAR-2018	101	20
103	Meena	DEVELOPER	6500	(null)	20-JUL-2019	101	20
104	Kiran	SALESMAN	4000	500	05-FEB-2017	105	10
105	Sana	MANAGER	8000	(null)	12-NOV-2014	(null)	10
106	John	SALESMAN	4500	300	25-AUG-2020	105	10
107	Priya	CLERK	3000	(null)	01-JUN-2021	108	30
108	Omar	MANAGER	7000	(null)	30-SEP-2016	(null)	30
109	Neha	DEVELOPER	6000	(null)	18-DEC-2022	101	20
110	Tina	ANALYST	5000	(null)	09-APR-2023	(null)	(null)

Table 3: PROJECTS

PROJECT_ID	PROJECT_NAME	DEPT_ID
1	Website Revamp	20
2	Mobile App	20
3	Ad Campaign	10
4	Hiring Drive	30

TIP: Look closely: Tina has no department (NULL), and the Finance department (40) has no employees. This is on purpose. It helps us understand outer joins later. Also, MANAGER_ID says who is the boss of that employee.

Script to create the tables (run this in Oracle)

```
CREATE TABLE departments (  
  dept_id NUMBER PRIMARY KEY, dept_name VARCHAR2(30), location VARCHAR2(30)  
);  
CREATE TABLE employees (  
  emp_id NUMBER PRIMARY KEY, emp_name VARCHAR2(30), job VARCHAR2(20),  
  salary NUMBER, commission NUMBER, hire_date DATE, manager_id NUMBER, dept_id NUMBER  
);  
CREATE TABLE projects (  
  project_id NUMBER PRIMARY KEY, project_name VARCHAR2(30), dept_id NUMBER  
);  
  
INSERT INTO departments VALUES (10, 'Sales', 'Mumbai');  
INSERT INTO departments VALUES (20, 'IT', 'Pune');  
INSERT INTO departments VALUES (30, 'HR', 'Delhi');  
INSERT INTO departments VALUES (40, 'Finance', 'Chennai');  
INSERT INTO employees VALUES  
  (101, 'Asha', 'MANAGER', 9000, NULL, TO_DATE('10-JAN-2015', 'DD-MON-YYYY'), NULL, 20);  
INSERT INTO employees VALUES  
  (102, 'Ravi', 'DEVELOPER', 6000, NULL, TO_DATE('15-MAR-2018', 'DD-MON-YYYY'), 101, 20);  
INSERT INTO employees VALUES  
  (103, 'Meena', 'DEVELOPER', 6500, NULL, TO_DATE('20-JUL-2019', 'DD-MON-YYYY'), 101, 20);  
INSERT INTO employees VALUES  
  (104, 'Kiran', 'SALESMAN', 4000, 500, TO_DATE('05-FEB-2017', 'DD-MON-YYYY'), 105, 10);  
INSERT INTO employees VALUES  
  (105, 'Sana', 'MANAGER', 8000, NULL, TO_DATE('12-NOV-2014', 'DD-MON-YYYY'), NULL, 10);  
INSERT INTO employees VALUES  
  (106, 'John', 'SALESMAN', 4500, 300, TO_DATE('25-AUG-2020', 'DD-MON-YYYY'), 105, 10);  
INSERT INTO employees VALUES  
  (107, 'Priya', 'CLERK', 3000, NULL, TO_DATE('01-JUN-2021', 'DD-MON-YYYY'), 108, 30);  
INSERT INTO employees VALUES  
  (108, 'Omar', 'MANAGER', 7000, NULL, TO_DATE('30-SEP-2016', 'DD-MON-YYYY'), NULL, 30);  
INSERT INTO employees VALUES  
  (109, 'Neha', 'DEVELOPER', 6000, NULL, TO_DATE('18-DEC-2022', 'DD-MON-YYYY'), 101, 20);  
INSERT INTO employees VALUES  
  (110, 'Tina', 'ANALYST', 5000, NULL, TO_DATE('09-APR-2023', 'DD-MON-YYYY'), NULL, NULL);  
INSERT INTO projects VALUES (1, 'Website Revamp', 20);  
INSERT INTO projects VALUES (2, 'Mobile App', 20);  
INSERT INTO projects VALUES (3, 'Ad Campaign', 10);  
INSERT INTO projects VALUES (4, 'Hiring Drive', 30);  
COMMIT;
```

Oracle rules to remember

- Every SQL statement ends with a semicolon ;.
- Keywords (SELECT, FROM, WHERE) are **not** case sensitive. But text inside quotes **is** case sensitive: 'Asha' is not 'ASHA'.
- Text and dates go inside **single quotes 'like this'**. Double quotes are only for column names with spaces.
- Oracle has a special dummy table called **DUAL**. We use it to test things like **SELECT 5+3 FROM dual;**.
- Oracle has no LIMIT keyword. Use **FETCH FIRST n ROWS ONLY** (Oracle 12c and above).
- Comments: **-- single line** and **/* many lines */**.

SQL Basics

SQL means **Structured Query Language**. A "query" is just a question you ask the database, like "Show me all employees who earn more than 6000."

1.1 SELECT ... FROM

SELECT tells Oracle **which columns** you want to see. **FROM** tells Oracle **which table** to look in.

ANALOGY: You open an Excel sheet and hide all columns except the ones you want. SELECT picks the columns you keep visible.

SYNTAX (the pattern)

```
SELECT column1, column2 FROM table_name;
```

EXAMPLE

```
SELECT dept_id, dept_name FROM departments;
```

RESULT

DEPT_ID	DEPT_NAME
10	Sales
20	IT
30	HR
40	Finance

Use a star ***** to see **all columns**: **SELECT * FROM employees;** This shows the whole EMPLOYEES table.

Column alias and simple maths

You can rename a column in the output using **AS**. You can also do maths on columns.

EXAMPLE

```
SELECT emp_name, salary * 12 AS yearly_salary FROM employees;
```

RESULT

EMP_NAME	YEARLY_SALARY
Asha	108000
Ravi	72000
Meena	78000
Kiran	48000
Sana	96000
John	54000
Priya	36000
Omar	84000
Neha	72000
Tina	60000

1.2 WHERE (filter rows)

WHERE keeps only the rows that match your condition. Rows that do not match are thrown away.

ANALOGY: WHERE is like a security guard at a gate. He checks each person (row). Only people who pass the check can enter the result.

SYNTAX (the pattern)

```
SELECT columns FROM table WHERE condition;
```

Operator	Meaning
=	equal to
<> or !=	not equal to
> <	greater than, less than
>= <=	greater than or equal, less than or equal

EXAMPLE

```
SELECT emp_name, salary FROM employees WHERE salary > 6000;
```

RESULT

EMP_NAME	SALARY
Asha	9000
Meena	6500
Sana	8000
Omar	7000

EXAMPLE

```
SELECT emp_name, job FROM employees WHERE job = 'MANAGER';
```

RESULT

EMP_NAME	JOB
Asha	MANAGER
Sana	MANAGER
Omar	MANAGER

WATCH OUT: Text values must be in single quotes, and the case must match. `job = 'manager'` will return no rows because the data is stored as MANAGER.

1.3 AND, OR, NOT

Use these to join more than one condition.

- **AND** : **both** conditions must be true.
- **OR** : **at least one** condition must be true.
- **NOT** : flips the condition (true becomes false).

ANALOGY: AND: to enter a cinema you need a ticket AND an ID card. OR: you can pay by cash OR by card. NOT: "anyone who is NOT a child."

EXAMPLE

```
SELECT emp_name, salary FROM employees WHERE dept_id = 20 AND salary > 6000;
```

RESULT

EMP_NAME	SALARY
Asha	9000
Meena	6500

EXAMPLE

```
SELECT emp_name, job FROM employees WHERE job = 'CLERK' OR job = 'ANALYST';
```

RESULT

EMP_NAME	JOB
Priya	CLERK
Tina	ANALYST

EXAMPLE

```
SELECT emp_name, dept_id FROM employees WHERE NOT dept_id = 20;
```

RESULT

EMP_NAME	DEPT_ID
Kiran	10
Sana	10
John	10
Priya	30
Omar	30

Notice Tina is missing. Her dept_id is NULL, and NULL is never "not equal" to 20 in SQL. It is simply unknown, so she is skipped.

Order matters: AND runs before OR

If you mix AND and OR, Oracle runs AND first. Use **brackets** to make your meaning clear.

EXAMPLE

```
SELECT emp_name, dept_id, salary FROM employees  
WHERE dept_id = 10 OR dept_id = 30 AND salary > 5000;
```

RESULT

EMP_NAME	DEPT_ID	SALARY
Kiran	10	4000
Sana	10	8000
John	10	4500
Omar	30	7000

Oracle read this as: (dept 10) OR (dept 30 AND salary above 5000). Now with brackets:

EXAMPLE

```
SELECT emp_name, dept_id, salary FROM employees
WHERE (dept_id = 10 OR dept_id = 30) AND salary > 5000;
```

RESULT

EMP_NAME	DEPT_ID	SALARY
Sana	10	8000
Omar	30	7000

1.4 BETWEEN

BETWEEN low AND high checks if a value is inside a range. Both ends are **included**.

EXAMPLE

```
SELECT emp_name, salary FROM employees WHERE salary BETWEEN 4000 AND 6000;
```

RESULT

EMP_NAME	SALARY
Ravi	6000
Kiran	4000
John	4500
Neha	6000
Tina	5000

Kiran (4000) and Ravi (6000) are in the result because the end values are included. It works for dates too.

1.5 IN

IN checks if a value matches **any value in a list**. It is a short way to write many ORs.

EXAMPLE

```
SELECT emp_name, dept_id FROM employees WHERE dept_id IN (10, 30);
```

RESULT

EMP_NAME	DEPT_ID
Kiran	10
Sana	10
John	10
Priya	30
Omar	30

This is the same as **dept_id = 10 OR dept_id = 30**. You can also write **NOT IN**.

1.6 LIKE (pattern search)

LIKE searches text by pattern. It has two special symbols:

Symbol	Meaning	Example
%	any number of characters (even zero)	' S %' starts with S
_	exactly one character	'_ a %' second letter is a
'%a'	ends with a	Asha, Meena, Sana ...
'%an%'	contains "an" anywhere	Kiran, Sana

ANALOGY: Imagine searching your phone contacts. Typing "Ra" and seeing every name that starts with Ra is like LIKE 'Ra%'.

EXAMPLE

```
SELECT emp_name FROM employees WHERE emp_name LIKE '%an%';
```

RESULT

EMP_NAME
Kiran
Sana

EXAMPLE

```
SELECT emp_name FROM employees WHERE emp_name LIKE '_a%';
```

RESULT

EMP_NAME
Ravi
Sana

Only Ravi and Sana have "a" as the second letter.

TIP: LIKE is case sensitive in Oracle. **LIKE 's%'** will not find Sana. Use **LOWER(emp_name) LIKE 's%'** to ignore case.

1.7 IS NULL / IS NOT NULL

NULL means "no value". You cannot check it with **=**. You must use **IS NULL** or **IS NOT NULL**.

ANALOGY: An empty box is not the same as a box with zero written on it. NULL is the empty box: we simply do not know the value.

EXAMPLE

```
SELECT emp_name, commission FROM employees WHERE commission IS NOT NULL;
```

RESULT

EMP_NAME	COMMISSION
Kiran	500
John	300

EXAMPLE

```
SELECT emp_name FROM employees WHERE commission IS NULL;
```

RESULT

EMP_NAME
Asha
Ravi
Meena
Sana
Priya
Omar
Neha
Tina

WATCH OUT: **WHERE commission = NULL** never returns anything. Always write **IS NULL**. Also, any maths with NULL gives NULL: **5000 + NULL** is NULL.

1.8 ORDER BY (sorting)

ORDER BY sorts the result. **ASC** means small to big (default). **DESC** means big to small. It is always the **last** line of a query.

EXAMPLE

```
SELECT emp_name, salary FROM employees ORDER BY salary DESC;
```

RESULT

EMP_NAME	SALARY
Asha	9000
Sana	8000
Omar	7000
Meena	6500
Ravi	6000
Neha	6000
Tina	5000
John	4500
Kiran	4000
Priya	3000

Ravi and Neha have the same salary, so Oracle may show them in either order. You can sort by more than one column:

EXAMPLE

```
SELECT emp_name, dept_id FROM employees ORDER BY dept_id, emp_name;
```

RESULT

EMP_NAME	DEPT_ID
John	10
Kiran	10
Sana	10
Asha	20
Meena	20
Neha	20
Ravi	20
Omar	30
Priya	30
Tina	(null)

First by dept_id, and inside each dept by name. In Oracle, NULLs come **last** in ASC order and **first** in DESC order. You can change this with **NULLS FIRST** or **NULLS LAST**.

Top 3 rows (Oracle way, since there is no LIMIT):

EXAMPLE

```
SELECT emp_name, salary FROM employees  
ORDER BY salary DESC  
FETCH FIRST 3 ROWS ONLY;
```

RESULT

EMP_NAME	SALARY
Asha	9000
Sana	8000
Omar	7000

1.9 DISTINCT (remove duplicates)

DISTINCT shows each value only once. Many employees are DEVELOPERS, but DISTINCT lists DEVELOPER only one time.

EXAMPLE

```
SELECT DISTINCT job FROM employees;
```

RESULT

JOB
MANAGER
DEVELOPER
SALESMAN
CLERK
ANALYST

The row order may be different on your system. Without DISTINCT you would see 10 rows.

Aggregate Queries

An **aggregate function** looks at many rows and gives you **one answer**. For example: total of all salaries, or the highest salary.

ANALOGY: A teacher has the marks of 40 students. She calculates the total, the average, the highest and the lowest. Those four calculations are aggregate functions.

Function	What it does
COUNT ()	counts rows
SUM ()	adds up numbers
AVG ()	finds the average
MAX ()	finds the biggest value
MIN ()	finds the smallest value

2.1 COUNT, SUM, AVG, MAX, MIN

EXAMPLE

```
SELECT COUNT(*) AS total_emp,
       SUM(salary) AS total_salary,
       AVG(salary) AS avg_salary,
       MAX(salary) AS highest,
       MIN(salary) AS lowest
FROM employees;
```

RESULT

TOTAL_EMP	TOTAL_SALARY	AVG_SALARY	HIGHEST	LOWEST
10	59000	5900	9000	3000

The whole table became **one row**. That is what aggregates do.

COUNT in three flavours

EXAMPLE

```
SELECT COUNT(*), COUNT(commission), COUNT(DISTINCT dept_id) FROM employees;
```

RESULT

COUNT(*)	COUNT(COMMISSION)	COUNT(DISTINCT DEPT_ID)
10	2	3

- **COUNT (*)** counts all rows: 10.
- **COUNT (column)** counts only rows where that column is **not NULL**: only 2 people have commission.
- **COUNT (DISTINCT column)** counts different values only: departments 10, 20, 30 = 3 (NULL is ignored).

WATCH OUT: All aggregate functions **ignore NULL** (except COUNT(*)). So **AVG (commission)** is $(500 + 300) / 2 = 400$, not $800 / 10$. If you want NULL treated as 0, use NVL: **AVG (NVL (commission, 0))** gives 80.

2.2 GROUP BY

GROUP BY splits the rows into groups, and then the aggregate function works **on each group separately**.

ANALOGY: Put all students into their class sections (A, B, C). Then find the average marks of each section. GROUP BY does the sectioning.

SYNTAX (the pattern)

```
SELECT group_column, AGGREGATE(column)
FROM table
GROUP BY group_column;
```

EXAMPLE

```
SELECT dept_id, COUNT(*) AS total_emp, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
ORDER BY dept_id;
```

RESULT

DEPT_ID	TOTAL_EMP	AVG_SALARY
10	3	5500
20	4	6875
30	2	5000
(null)	1	5000

Tina (NULL department) forms her own group.

WATCH OUT: Golden rule: every column in SELECT that is **not** inside an aggregate function must be written in GROUP BY. If you forget, Oracle gives error ORA-00979: not a GROUP BY expression.

2.3 HAVING (filter the groups)

WHERE filters **rows before** grouping. **HAVING** filters **groups after** grouping. You cannot use aggregates like COUNT() in WHERE.

	WHERE	HAVING
Works on	individual rows	groups
Runs	before GROUP BY	after GROUP BY
Can use COUNT/SUM?	No	Yes

EXAMPLE

```
SELECT dept_id, COUNT(*) AS total_emp
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 2
ORDER BY dept_id;
```

RESULT

DEPT_ID	TOTAL_EMP
10	3
20	4

Only groups with more than 2 employees stay.

Using both together: "Among employees earning more than 4000, which departments have at least 2 such employees?"

EXAMPLE

```
SELECT dept_id, COUNT(*) AS total_emp
FROM employees
WHERE salary > 4000
GROUP BY dept_id
HAVING COUNT(*) >= 2
ORDER BY dept_id;
```

RESULT

DEPT_ID	TOTAL_EMP
10	2
20	4

Step by step: WHERE removes Kiran, Priya (4000 or below), then rows are grouped, then HAVING keeps the big groups.

The order Oracle runs a query (very important!)

1. FROM (pick the table)
2. WHERE (filter rows)
3. GROUP BY (make groups)
4. HAVING (filter groups)
5. SELECT (choose columns)
6. ORDER BY (sort the result)

This is why you cannot use a column alias from SELECT inside WHERE: WHERE runs before SELECT. ORDER BY is the only place where the alias works.

Joins

Data is kept in separate tables to avoid repeating things. Employee rows only store **dept_id** (like 20), not the name "IT". A **join** glues two or more tables together using a common column, so you can see the name "IT" next to each employee.

ANALOGY: You have a student list with a class code (like "C1") and another list that says "C1 = Science". To print student names with class names, you match the codes. That matching is a join.

Table alias (a short nickname)

In joins we give tables short nicknames: **employees e**. Then we write **e.emp_name** to say "emp_name from the table e". This avoids confusion when both tables have a column with the same name (like dept_id).

Join type	What you get
Inner join	only rows that match in both tables
Left outer join	all rows from the left table + matches from the right
Right outer join	all rows from the right table + matches from the left
Full outer join	all rows from both tables, matched where possible
Self join	a table joined with itself
Equi join	any join that uses = in the condition
Natural join	automatic join on columns with the same name

3.1 Inner join

SYNTAX (the pattern)

```
SELECT a.col, b.col
FROM table_a a
INNER JOIN table_b b ON a.common_col = b.common_col;
```

EXAMPLE

```
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

RESULT

EMP_NAME	DEPT_NAME
Asha	IT
Ravi	IT
Meena	IT
Kiran	Sales
Sana	Sales
John	Sales
Priya	HR
Omar	HR
Neha	IT

Only 9 rows. Tina is missing (no department) and Finance is missing (no employees). Inner join keeps only the matches. **JOIN** alone means **INNER JOIN**.

3.2 Equi join (the old Oracle style)

An equi join is any join that uses the **equals (=)** sign. Older Oracle code writes it with a comma and puts the condition in WHERE. The result is the same as the inner join above.

EXAMPLE

```
SELECT e.emp_name, d.dept_name
FROM employees e, departments d
WHERE e.dept_id = d.dept_id;
```

RESULT

EMP_NAME	DEPT_NAME
Asha	IT
Ravi	IT
Meena	IT
Kiran	Sales
Sana	Sales
John	Sales
Priya	HR
Omar	HR
Neha	IT

Same 9 rows. If you forget the WHERE line, every employee is paired with every department (10 x 4 = 40 rows). This mistake is called a **Cartesian product**.

TIP: A join that uses something other than = (like BETWEEN or >) is called a **non-equi join**.

3.3 Natural join

Natural join looks for columns with the **same name** in both tables and joins on them automatically. EMPLOYEES and DEPARTMENTS share only dept_id, so it works.

EXAMPLE

```
SELECT emp_name, dept_name
FROM employees NATURAL JOIN departments;
```

RESULT

EMP_NAME	DEPT_NAME
Asha	IT
Ravi	IT
Meena	IT
Kiran	Sales
Sana	Sales
John	Sales
Priya	HR
Omar	HR
Neha	IT

WATCH OUT: Do not write a table prefix on the common column (no **e.dept_id**) in a natural join. Also it is risky: if someone adds another same-named column later, the result silently changes. Most people prefer **JOIN ... ON**.

3.4 Self join

A self join joins a table **with itself**. In EMPLOYEES, the column manager_id holds the emp_id of the boss. Both employee and boss are in the same table, so we use the table twice with two different nicknames.

ANALOGY: Imagine holding the same class list in both hands. The left list is "students" and the right list is "teachers", but it is the same paper.

EXAMPLE

```
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id;
```

RESULT

EMPLOYEE	MANAGER
Ravi	Asha
Meena	Asha
Kiran	Sana
John	Sana
Priya	Omar
Neha	Asha

Asha, Sana, Omar and Tina have NULL manager_id, so they do not appear as employees here (inner join).

3.5 Outer joins

Inner join throws away rows with no match. Outer joins **keep** them and fill the missing side with NULL.

Left outer join: keep everything from the left table

EXAMPLE

```
SELECT e.emp_name, d.dept_name
FROM employees e
LEFT OUTER JOIN departments d ON e.dept_id = d.dept_id;
```

RESULT

EMP_NAME	DEPT_NAME
Asha	IT
Ravi	IT
Meena	IT
Kiran	Sales
Sana	Sales
John	Sales
Priya	HR
Omar	HR
Neha	IT
Tina	(null)

All 10 employees are shown. Tina appears with a NULL department.

Right outer join: keep everything from the right table

EXAMPLE

```
SELECT d.dept_name, e.emp_name
FROM employees e
RIGHT OUTER JOIN departments d ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

RESULT

DEPT_NAME	EMP_NAME
Finance	(null)

Without the WHERE line you get the 9 matches plus Finance with a NULL employee. The WHERE line keeps only departments that have **no employees**. This is a very common interview trick.

Full outer join: keep everything from both tables

EXAMPLE

```
SELECT e.emp_name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;
```

RESULT

EMP_NAME	DEPT_NAME
...9 matching rows...	...
Tina	(null)
(null)	Finance

Total 11 rows: 9 matches + Tina (no dept) + Finance (no employees).

Oracle old style: the (+) sign

Old Oracle code writes outer joins with **(+)** on the side that may have missing rows. Both queries below do a left outer join.

```
-- new style
SELECT e.emp_name, d.dept_name
FROM employees e LEFT JOIN departments d ON e.dept_id = d.dept_id;

-- old Oracle style
SELECT e.emp_name, d.dept_name
FROM employees e, departments d
WHERE e.dept_id = d.dept_id(+);
```

3.6 Joining 3 or more tables

Just keep adding **JOIN ... ON ...** lines. Think of it as a chain: EMPLOYEES to DEPARTMENTS to PROJECTS.

EXAMPLE

```
SELECT e.emp_name, d.dept_name, p.project_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
JOIN projects p    ON d.dept_id = p.dept_id
WHERE d.dept_name = 'Sales';
```

RESULT

EMP_NAME	DEPT_NAME	PROJECT_NAME
Kiran	Sales	Ad Campaign
Sana	Sales	Ad Campaign
John	Sales	Ad Campaign

Sales has 3 employees and 1 project, so we get $3 \times 1 = 3$ rows. If you change the filter to 'IT' (4 employees, 2 projects) you get $4 \times 2 = 8$ rows. Rule: **for N tables you need N - 1 join conditions.**

Subqueries

A **subquery** is a query written **inside another query**, in brackets. The inside query (inner) gives an answer, and the outside query (outer) uses that answer.

ANALOGY: You want a phone that costs more than your friend's phone. First you ask: "What is the price of your phone?" (inner question). Then you search for phones above that price (outer question).

Type	Meaning
Single-row	inner query returns one value. Use =, >, <
Multi-row	inner query returns many values. Use IN, ANY, ALL
Correlated	inner query uses a column of the outer query and runs again for each outer row
In FROM	inner query acts like a temporary table (inline view)
In SELECT	inner query gives one value in a column (scalar subquery)

4.1 Single-row subquery

EXAMPLE

```
SELECT emp_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

RESULT

EMP_NAME	SALARY
Asha	9000
Ravi	6000
Meena	6500
Sana	8000
Omar	7000
Neha	6000

The inner query gives 5900 (the average). Then the outer query becomes: salary > 5900. Next: who earns the highest salary?

EXAMPLE

```
SELECT emp_name, salary
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

RESULT

EMP_NAME	SALARY
Asha	9000

4.2 Multi-row subquery with IN

When the inner query returns **many values**, = does not work. Use **IN**.

EXAMPLE

```
SELECT emp_name, dept_id
FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments
                  WHERE location = 'Pune');
```

RESULT

EMP_NAME	DEPT_ID
Asha	20
Ravi	20
Meena	20
Neha	20

Inner query returns department 20 (Pune). Outer query returns the employees of that department.

WATCH OUT: NOT IN has a trap: if the inner list contains even one NULL, **NOT IN** returns **no rows at all**. Add **WHERE column IS NOT NULL** inside the subquery to be safe.

4.3 ANY and ALL

Both compare a value with a list. The department 30 salaries are 3000 and 7000.

- **> ANY (3000, 7000)** means bigger than **at least one** value, so bigger than 3000 (the smallest).
- **< ANY (3000, 7000)** means smaller than at least one, so smaller than 7000 (the biggest).
- **> ALL (3000, 7000)** means bigger than **every** value, so bigger than 7000.
- **< ALL (3000, 7000)** means smaller than every value, so smaller than 3000.
- **= ANY** is the same as **IN**.

ANALOGY: ANY is like "you only need to beat one player to qualify." ALL is like "you must beat every player."

EXAMPLE

```
SELECT emp_name, salary
FROM employees
WHERE salary < ANY (SELECT salary FROM employees WHERE dept_id = 30);
```

RESULT

EMP_NAME	SALARY
Ravi	6000
Meena	6500
Kiran	4000
John	4500
Priya	3000
Neha	6000
Tina	5000

Everyone who earns less than 7000.

EXAMPLE

```
SELECT emp_name, salary
FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id = 30);
```

RESULT

EMP_NAME	SALARY
Asha	9000
Sana	8000

Only people who earn more than 7000 (higher than everyone in dept 30).

4.4 Correlated subquery

In a normal subquery, the inner query runs **once**. In a correlated subquery, the inner query uses a value from the outer row, so it runs **again for every outer row**.

ANALOGY: A teacher asks each student: "Are your marks above the average of **your own** class?" Each student compares with their own class average, so the answer is worked out again for each student.

EXAMPLE

```
SELECT e.emp_name, e.salary, e.dept_id
FROM employees e
WHERE e.salary > (SELECT AVG(x.salary)
                  FROM employees x
                  WHERE x.dept_id = e.dept_id);
```

RESULT

EMP_NAME	SALARY	DEPT_ID
Asha	9000	20
Sana	8000	10
Omar	7000	30

Department averages: 10 is 5500, 20 is 6875, 30 is 5000. So Sana, Asha and Omar earn above their own department average. Tina is skipped because her department is NULL.

EXISTS: a popular correlated subquery

EXISTS is true if the inner query finds **at least one row**. It only checks "is there something?" and does not care about the values.

EXAMPLE

```
SELECT d.dept_id, d.dept_name
FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e
              WHERE e.dept_id = d.dept_id)
ORDER BY d.dept_id;
```

RESULT

DEPT_ID	DEPT_NAME
10	Sales
20	IT
30	HR

Finance is left out because it has no employees.

4.5 Subquery in FROM (inline view)

The inner query creates a **temporary table** that the outer query can use. You must give it a nickname (here **t**).

EXAMPLE

```
SELECT d.dept_name, t.avg_sal
FROM (SELECT dept_id, AVG(salary) AS avg_sal
      FROM employees
      GROUP BY dept_id) t
JOIN departments d ON d.dept_id = t.dept_id
ORDER BY d.dept_id;
```

RESULT

DEPT_NAME	AVG_SAL
Sales	5500
IT	6875
HR	5000

4.6 Subquery in SELECT (scalar subquery)

A subquery inside the SELECT list must return **exactly one value**. That value is shown as a column.

EXAMPLE

```
SELECT emp_name, salary,
       (SELECT AVG(salary) FROM employees) AS company_avg
FROM employees
WHERE dept_id = 30;
```

RESULT

EMP_NAME	SALARY	COMPANY_AVG
Priya	3000	5900
Omar	7000	5900

Set Operations

Set operations **combine the results of two queries** one below the other (stacking). Joins put tables side by side; set operations stack results on top of each other.

ANALOGY: Two clubs have their own member lists. UNION = everyone from both clubs (each person once). INTERSECT = people who are in both clubs. MINUS = people in club 1 who are not in club 2.

Operator	Result
UNION	all rows from both, duplicates removed
UNION ALL	all rows from both, duplicates kept (faster)
INTERSECT	only rows that appear in both
MINUS	rows in the first query that are not in the second

Rules

- Both queries must have the **same number of columns**.
- Matching columns must have **compatible data types** (number with number, text with text).
- Column names in the result come from the **first** query.
- ORDER BY is allowed only once, at the very end.

We use dept_id from DEPARTMENTS (10, 20, 30, 40), from PROJECTS (20, 20, 10, 30) and from EMPLOYEES (10, 20, 30, and Tina's NULL).

5.1 UNION

EXAMPLE

```
SELECT dept_id FROM projects
UNION
SELECT dept_id FROM departments;
```

RESULT

DEPT_ID
10
20
30
40

Projects gave 4 rows, departments gave 4 rows, but duplicates are removed, so 4 rows remain. The result is also sorted.

5.2 UNION ALL

EXAMPLE

```
SELECT dept_id FROM projects
UNION ALL
SELECT dept_id FROM departments;
```

RESULT

DEPT_ID
20
20
10
30
10
20
30
40

All 8 rows are kept, including duplicates. It does not sort, so it is faster.

5.3 INTERSECT

EXAMPLE

```
SELECT dept_id FROM departments
INTERSECT
SELECT dept_id FROM employees;
```

RESULT

DEPT_ID
10
20
30

Departments that also have employees. 40 is not in EMPLOYEES, so it is gone.

5.4 MINUS (Oracle only word)

EXAMPLE

```
SELECT dept_id FROM departments
MINUS
SELECT dept_id FROM employees;
```

RESULT

DEPT_ID
40

Departments with no employees. Order matters: swap the two queries and you get a single NULL row instead (Tina's missing department). In other databases (like PostgreSQL) MINUS is called **EXCEPT**.

Important Oracle-Specific Things

These functions are used again and again in Oracle work and interviews. Each one is small and easy once you see an example.

6.1 NVL (replace NULL)

NVL(value, replacement) returns the replacement if the value is NULL. Otherwise it returns the value itself.

ANALOGY: If a student's phone number box is empty, you write "Not given". NVL does that for NULL.

EXAMPLE

```
SELECT emp_name, salary, commission,
       salary + NVL(commission, 0) AS total_pay
FROM employees
WHERE dept_id = 10;
```

RESULT

EMP_NAME	SALARY	COMMISSION	TOTAL_PAY
Kiran	4000	500	4500
Sana	8000	(null)	8000
John	4500	300	4800

Without NVL, Sana's total pay would be NULL because $8000 + \text{NULL} = \text{NULL}$.

TIP: **NVL2(x, if_not_null, if_null)** picks between two answers. Example: **NVL2(commission, 'Has commission', 'No commission')**. **COALESCE(a, b, c)** returns the first non-NULL value in the list.

6.2 DECODE (Oracle's if-else)

DECODE(column, value1, result1, value2, result2, ..., default) compares a column with values one by one, like a small if-else chain. It only checks **equality**.

EXAMPLE

```
SELECT emp_name, job,
       DECODE(job, 'MANAGER', 'Boss',
              'DEVELOPER', 'Coder',
              'Other') AS role
FROM employees
WHERE dept_id = 20;
```

RESULT

EMP_NAME	JOB	ROLE
Asha	MANAGER	Boss
Ravi	DEVELOPER	Coder
Meena	DEVELOPER	Coder
Neha	DEVELOPER	Coder

6.3 CASE (the modern, powerful if-else)

CASE works like **DECODE** but is more flexible: it can use **>**, **<**, **BETWEEN**, **LIKE** and more. It is also standard SQL, so it works in other databases too.

SYNTAX (the pattern)

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE default_result
END
```

EXAMPLE

```
SELECT emp_name, salary,
       CASE WHEN salary >= 8000 THEN 'High'
            WHEN salary >= 5000 THEN 'Medium'
            ELSE 'Low'
       END AS salary_level
FROM employees
WHERE dept_id IN (10, 30)
ORDER BY salary DESC;
```

RESULT

EMP_NAME	SALARY	SALARY_LEVEL
Sana	8000	High
Omar	7000	Medium
John	4500	Low
Kiran	4000	Low
Priya	3000	Low

Oracle checks the **WHEN** lines from top to bottom and stops at the first true one.

6.4 SYSDATE and DUAL

SYSDATE gives the current date and time of the Oracle server. **DUAL** is a one-row dummy table used when you need to run a function without any real table.

EXAMPLE

```
SELECT SYSDATE FROM dual;
```

RESULT

SYSDATE
28-SEP-26

Your result will show your own today's date. Oracle shows only the date part by default, but the time is stored too.

Date maths is easy: **date + 1 = tomorrow**, **date - 1 = yesterday**, and **date - date = number of days**.

EXAMPLE

```
SELECT TO_DATE('01-JAN-2026','DD-MON-YYYY') + 10 AS plus_10_days,
       TO_DATE('10-JAN-2026','DD-MON-YYYY') - TO_DATE('01-JAN-2026','DD-MON-YYYY') AS days_between
FROM dual;
```

RESULT

PLUS_10_DAYS	DAYS_BETWEEN
11-JAN-26	9

6.5 TO_DATE (text to date)

TO_DATE(text, format) converts a text into a real DATE. The format tells Oracle how to read the text.

EXAMPLE

```
SELECT emp_name, hire_date
FROM employees
WHERE hire_date > TO_DATE('01-JAN-2020', 'DD-MON-YYYY');
```

RESULT

EMP_NAME	HIRE_DATE
John	25-AUG-20
Priya	01-JUN-21
Neha	18-DEC-22
Tina	09-APR-23

These are the employees hired after 1 January 2020.

WATCH OUT: Always give the format. If you write only **'01-01-2020'** without a format, Oracle guesses using your session settings, and this can fail on another computer.

6.6 TO_CHAR (date or number to text)

TO_CHAR(value, format) converts a date or number into text in the shape you want. It is mostly used to **show dates nicely**.

Format	Meaning	Example output
YYYY	4-digit year	2017
YY	2-digit year	17
MM	month number	02
MON	short month name	FEB
MONTH	full month name	FEBRUARY
DD	day of month	05
DAY	name of weekday	MONDAY
DY	short weekday	MON
HH24:MI:SS	time in 24-hour clock	14:30:00
FM	removes extra spaces and leading zeros	FMDD gives 5, not 05

EXAMPLE

```
SELECT emp_name,
       TO_CHAR(hire_date, 'FMDD Month YYYY') AS joined_on
FROM employees
WHERE dept_id = 10;
```

RESULT

EMP_NAME	JOINED_ON
Kiran	5 February 2017
Sana	12 November 2014
John	25 August 2020

For numbers, **9** is a digit place and **0** forces a zero. Example: **TO_CHAR(9000, '99,999.00')** gives **9,000.00**. You can also pull a single part out of a date with **EXTRACT** (see 6.8).

6.7 String functions

These change or inspect text. Try them on **dua1** first.

Function	Example	Result
UPPER(x)	UPPER('oracle')	ORACLE
LOWER(x)	LOWER('ORACLE')	oracle
INITCAP(x)	INITCAP('hello world')	Hello World
LENGTH(x)	LENGTH('Oracle')	6
SUBSTR(x, start, len)	SUBSTR('Oracle SQL', 1, 6)	Oracle
SUBSTR (negative start)	SUBSTR('Oracle SQL', -3)	SQL (counts from the end)
INSTR(x, find)	INSTR('Oracle SQL', 'SQL')	8 (position where it starts)
REPLACE(x, old, new)	REPLACE('Hello World', 'World', 'SQL')	Hello SQL
TRIM(x)	TRIM(' hi ')	hi (spaces removed both sides)
LTRIM / RTRIM	LTRIM('xxhi', 'x')	hi
LPAD(x, n, ch)	LPAD('7', 3, '0')	007
RPAD(x, n, ch)	RPAD('Hi', 5, '*')	Hi***
CONCAT(a, b)	CONCAT('Data', 'base')	Database
a b	'Hello' ' ' 'SQL'	Hello SQL

EXAMPLE

```
SELECT emp_name,  
       UPPER(emp_name)      AS upper_name,  
       LENGTH(emp_name)     AS len,  
       SUBSTR(emp_name,1,3) AS first3  
FROM employees  
WHERE dept_id = 30;
```

RESULT

EMP_NAME	UPPER_NAME	LEN	FIRST3
Priya	PRIYA	5	Pri
Omar	OMAR	4	Oma

EXAMPLE

```
SELECT emp_name || ' works as ' || job AS info  
FROM employees  
WHERE emp_id IN (101, 107);
```

RESULT

INFO
Asha works as MANAGER
Priya works as CLERK

The double bar **||** joins texts together (concatenation).

6.8 Date functions

Function	What it does	Example and result
SYSDATE	today's date and time	see 6.4
ADD_MONTHS(d, n)	adds n months to a date	ADD_MONTHS('15-JAN-2026', 3) gives 15-APR-26
MONTHS_BETWEEN(d1, d2)	months from d2 to d1 (can be decimal)	MONTHS_BETWEEN('01-MAR-2026', '01-JAN-2026') gives 2
LAST_DAY(d)	last date of that month	LAST_DAY('15-FEB-2026') gives 28-FEB-26
NEXT_DAY(d, day)	date of the next given weekday	NEXT_DAY('15-JAN-2026', 'MONDAY') gives 19-JAN-26
TRUNC(d, fmt)	cuts the date down (removes time, or goes to month/year start)	TRUNC('15-JAN-2026', 'MM') gives 01-JAN-26
ROUND(d, fmt)	rounds the date to the nearest month/year	ROUND('20-JAN-2026', 'MM') gives 01-FEB-26
EXTRACT(part FROM d)	takes YEAR, MONTH or DAY out of a date	EXTRACT(YEAR FROM hire_date) gives 2017

In the table the dates are written as short text to save space. In real queries, wrap them with **TO_DATE(...)**. Here is a real one:

EXAMPLE

```
SELECT ADD_MONTHS(TO_DATE('15-JAN-2026','DD-MON-YYYY'), 3) AS add_3_months,
       LAST_DAY(TO_DATE('15-FEB-2026','DD-MON-YYYY'))      AS month_end,
       NEXT_DAY(TO_DATE('15-JAN-2026','DD-MON-YYYY'),'MONDAY') AS next_monday
FROM dual;
```

RESULT

ADD_3_MONTHS	MONTH_END	NEXT_MONDAY
15-APR-26	28-FEB-26	19-JAN-26

EXAMPLE

```
SELECT emp_name,
       EXTRACT(YEAR FROM hire_date) AS join_year
FROM employees
WHERE dept_id = 30;
```

RESULT

EMP_NAME	JOIN_YEAR
Priya	2021
Omar	2016

Years of service is a classic question. Divide the months by 12 and cut off the decimal part with TRUNC:

EXAMPLE

```
SELECT emp_name,  
       TRUNC(MONTHS_BETWEEN(SYSDATE, hire_date) / 12) AS years_of_service  
FROM employees  
WHERE dept_id = 10;
```

RESULT

EMP_NAME	YEARS_OF_SERVICE
Kiran	9
Sana	11
John	6

Results shown are as on 28-SEP-2026. They change as time passes.

Cheat Sheet and Practice

Use this page for quick revision, then try the practice questions. Cover the answers and write the queries yourself first.

Full query pattern

```
SELECT  DISTINCT column, AGGREGATE(column)      -- what to show
FROM    table1 t1
JOIN    table2 t2 ON t1.col = t2.col           -- combine tables
WHERE   row_condition                         -- filter rows
GROUP BY column                               -- make groups
HAVING  group_condition                       -- filter groups
ORDER BY column DESC                          -- sort
FETCH FIRST 5 ROWS ONLY;                      -- limit rows
```

Quick memory tips

- NULL: always use **IS NULL**, never = NULL.
- Text: single quotes and exact case.
- Aggregate + normal column in SELECT: the normal column goes in **GROUP BY**.
- Filtering rows: **WHERE**. Filtering groups: **HAVING**.
- N tables need N - 1 join conditions.
- Many values from a subquery: use **IN / ANY / ALL**, not =.
- UNION removes duplicates. UNION ALL keeps them. MINUS is Oracle's "except".
- NVL fixes NULLs. DECODE and CASE are if-else. TO_CHAR shows, TO_DATE reads.

Practice questions (with answers)

Q1. Show employees hired after 31-DEC-2019, newest first.

```
SELECT emp_name, hire_date FROM employees
WHERE hire_date > TO_DATE('31-DEC-2019', 'DD-MON-YYYY')
ORDER BY hire_date DESC;
```

Q2. Show every department name with its number of employees, including departments that have none.

```
SELECT d.dept_name, COUNT(e.emp_id) AS total_emp
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

Q3. Find the second highest salary. (Answer is 8000.)

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Q4. List the names of employees who are managers of someone (self join).

```
SELECT DISTINCT m.emp_name
FROM employees e JOIN employees m ON e.manager_id = m.emp_id;
```

Q5. Show total pay (salary + commission) for each employee, treating missing commission as 0.

```
SELECT emp_name, salary + NVL(commission, 0) AS total_pay FROM employees;
```

Q6. Which departments have no projects? (Answer is 40.)

```
SELECT dept_id FROM departments
MINUS
SELECT dept_id FROM projects;
```

Q7. Show department names that have more than 2 employees.

```
SELECT d.dept_name, COUNT(*) AS total_emp
FROM employees e JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
HAVING COUNT(*) > 2;
```

Q8. Show employees who earn more than Ravi.

```
SELECT emp_name, salary FROM employees
WHERE salary > (SELECT salary FROM employees WHERE emp_name = 'Ravi');
```

Keep practising. Type every query yourself. That is the fastest way to learn SQL!